

Multiplicação de Matrizes

Autores: Juan Henriques Passos - 15464826
Fernando Valentim Torres - 15452340
Fellipe Catuzzi - 11871198

Introdução

O trabalho consiste na análise de 2 técnicas de multiplicação de matrizes, utilizando alocação estática ou dinâmica. Tal experimento, tem como fatores:

- Memória: Alocação dinâmica ou estática.
- Técnica: Multiplicação sem otimização, Loop Unrolling, Loop Interchange e Loop Tiling.

Os resultados a serem analisados:

1. **O tempo de execução;**
2. **L1-dcache-load;**
3. **L1-dcache-load-misses;**
4. **branch-instructions;**
5. **branch-misses.**

Estes resultados foram coletados utilizando o comando "perf stat". O código foi executado 10 vezes, e a partir dos dados coletados, foram calculadas a média e o intervalo de confiança de 95% para cada métrica.

Para a análise da cache, foram escolhidos a multiplicação normal e a blocagem, tendo em vista que apesar de aumentar os números de branches, se beneficia da definição da cache. E para análise da branch, foi escolhido a técnica unrolling, tendo em vista a diminuição do tamanho do loop, o que resulta em menos branches.

Média e Intervalo de Confiança

Dados apresentados no [CSV no Github](#).

- Tempos de Execução:

Tempo de resposta	Tempo de Resposta	
	Média	Intervalo de Confiança
1	0,3282462918	[0,31553776; 0,34095482]
2	0,3294646147	[0,31978636; 0,33914287]
5	0,255084702	[0,25038992; 0,25977949]
6	0,2467651080	[0,24214349; 0,25138672]

7	0,2903623688	[0,28458917; 0,29613557]
8	0,2846056780	[0,28367584; 0,28553552]

Os experimentos foram divididos conforme especificação:

1. Sem otimização (nem de compilação) com alocação estática
2. Sem otimização (nem de compilação) com alocação dinâmica
3. Loop Interchange com alocação estática
4. Loop Interchange com alocação dinâmica
5. Loop Unrolling com alocação estática
6. Loop Unrolling com alocação dinâmica
7. Loop Tiling com alocação estática
8. Loop Tiling com alocação dinâmica

Com base nos dados experimentais fornecidos, é possível extrair as seguintes conclusões sobre o tempo de resposta das diferentes configurações:

- Eficácia das Otimizações: Ambas as técnicas de otimização de laço testadas, Loop Unrolling e Loop Tiling, foram eficazes em reduzir o tempo de resposta quando comparadas aos experimentos sem otimização (Linha de Base). Qualquer uma das otimizações resultou em um ganho de desempenho.
- Superioridade do Loop Unrolling: A técnica de Loop Unrolling (Experimentos 5 e 6) demonstrou ser significativamente mais eficiente que o Loop Tiling. O tempo de resposta foi reduzido em aproximadamente 25% em comparação com a linha de base, enquanto o Loop Tiling (Experimentos 7 e 8) obteve uma redução de cerca de 13%.
- Impacto da Alocação de Memória:
 - Nos casos sem otimização, a diferença de desempenho entre a alocação estática (Exp. 1: 0,3282) e a dinâmica (Exp. 2: 0,3294) foi insignificante.
 - No entanto, quando as otimizações foram aplicadas, a alocação dinâmica consistentemente apresentou um desempenho ligeiramente superior ao da alocação estática. Isso foi observado tanto no Loop Unrolling (0,2467 vs. 0,2550) quanto no Loop Tiling (0,2846 vs. 0,2903).

A configuração mais performática foi a do Experimento 6, que combinou a técnica de otimização Loop Unrolling com alocação dinâmica, alcançando o menor tempo de resposta médio de 0,2467. Isso indica que, para o problema em questão, o Loop Unrolling é a estratégia de otimização de laço preferencial, e seu efeito é maximizado quando utilizado em conjunto com a alocação dinâmica de memória.

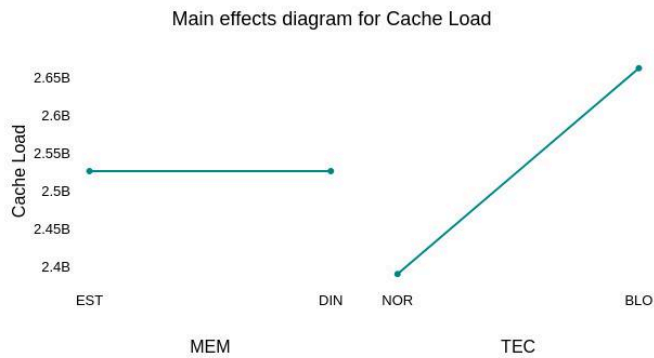
Análise dos experimentos

Cache	Cache Load		Cache Miss	
Experimento	Média	Intervalo de confiança	Média	Intervalo de Confiança
1	2.396.539.218	[2382880569,352; 2410197866,048]	79.612.483	[55524919,728; 103700047,072]
2	2.384.913.141	[2377426731,092; 2392399550,708]	88.618.562	[65899341,281; 111337783,319]
7	2.655.829.922	[2645333441,902; 2666326401,698]	7.458.064	[6749239,416; 8166889,384]
8	2.667.872.143	[2655284173,410; 2680460111,790]	7.434.604	[6777921,569; 8091285,631]

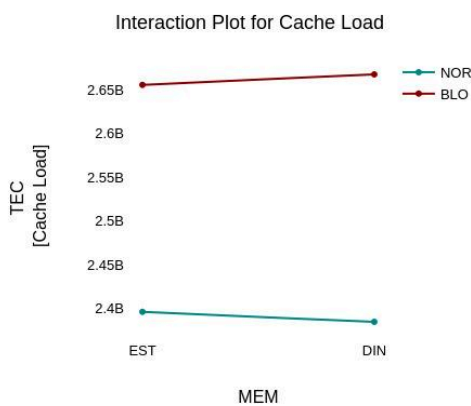
- **L1-dcache-load**

Cache Load		
Parâmetro	Média Estimada	Variação %
Q0	2526288606	
Qa	104036	0,00%
Qb	135562426,5	99,81%
Qab	5917074,5	0,19%

Main Effects



Interaction Effects



Conclusões

A análise dos fatores para a métrica Cache Load revela que a técnica de otimização (TEC) é o fator preponderante. A aplicação da blocagem (BLO) resulta em um aumento substancial do número de acessos à cache em comparação à versão não otimizada (NOR). Por outro lado, o tipo de alocação de memória (MEM) demonstrou ter um efeito praticamente nulo quando analisado isoladamente.

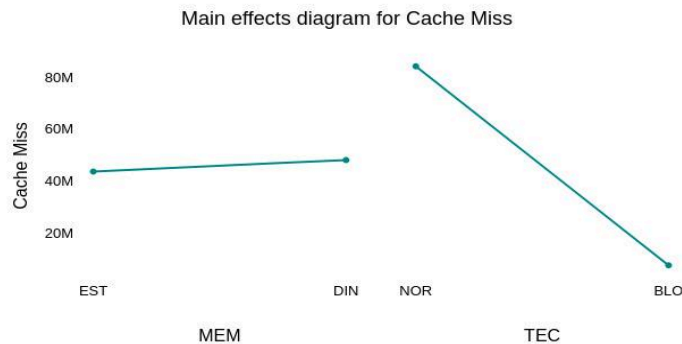
O gráfico de interação confirma essa tendência, mostrando que as linhas são quase paralelas. Isso indica que não há uma interação significativa entre os fatores, ou seja, o aumento de Cache Loads provocado pela blocagem ocorre praticamente independentemente de a alocação de memória ser estática ou dinâmica.

- **L1-dcache-load-misses**

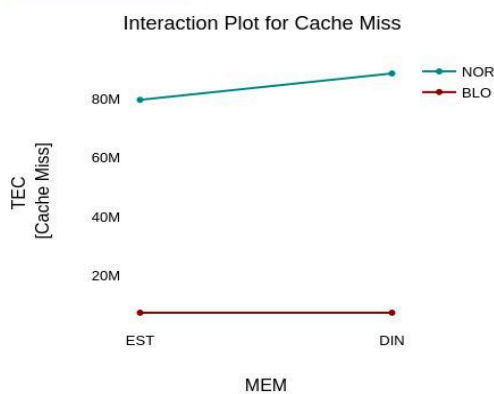
Cache Miss		
Parâmetro	Média Estimada	Variação %

Q0	45780928,25	
Qa	2245654,75	0,34%
Qb	-38334594,25	99,31%
Qab	-2257384,75	0,34%

Main Effects



Interaction Effects



Conclusões

Fica evidente, a partir da análise da tabela e da análise gráfica, que a técnica de otimização é o fator preponderante, com a aplicação da blocagem reduzindo o número de cache misses em quase 90% em comparação com a versão não otimizada. É nesta métrica que a relação entre o tempo de resposta e os eventos de hardware se torna clara. Embora a blocagem tenha aumentado o número de Cache Loads (como visto na análise anterior), ela o fez de forma a garantir a localidade de dados, o que resultou nesta redução massiva de Cache Misses. Como um *miss* na cache gera um tempo de espera muito maior para o processador do que um *load* bem-sucedido, esta troca é extremamente vantajosa.

Branch	Branch Instructions		Branch Miss	
Experimento	Média	Intervalo de confiança	Média	Intervalo de Confiança
1	207.373.486	[202784704,861; 211962266,339]	606.198	[580488,240; 631907,160]

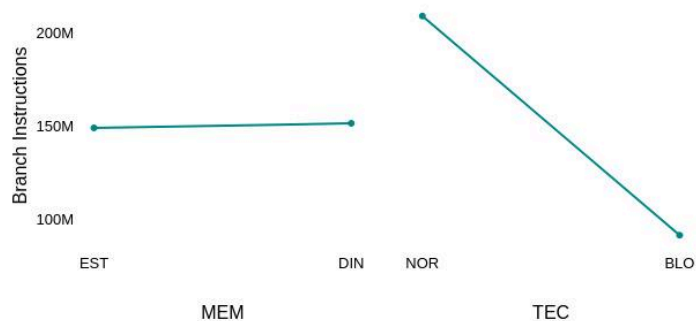
2	210.251.634	[207718050,4149100; 212785217,5850900]	622.534	[608192,7169981; 636875,4830019]
5	90.679.179	[85281251,279; 96077105,921]	511.886	[481549,746; 542223,054]
6	92.742.828	[89362174,935; 96123480,665]	575.983	[556637,508; 595327,692]

- **Branch-instructions**

Branch Instructions		
Parâmetro	Média Estimada	Variação %
Q0	150261781,8	
Qa	1235449,25	0,04%
Qb	-58550778,25	99,95%
Qab	-203624,75	0,00%

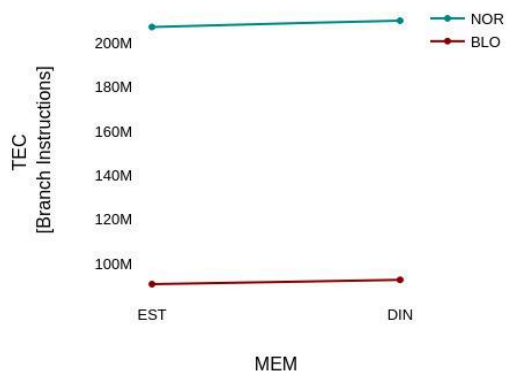
Main Effects

Main effects diagram for Branch Instructions



Interaction Effects

Interaction Plot for Branch Instructions



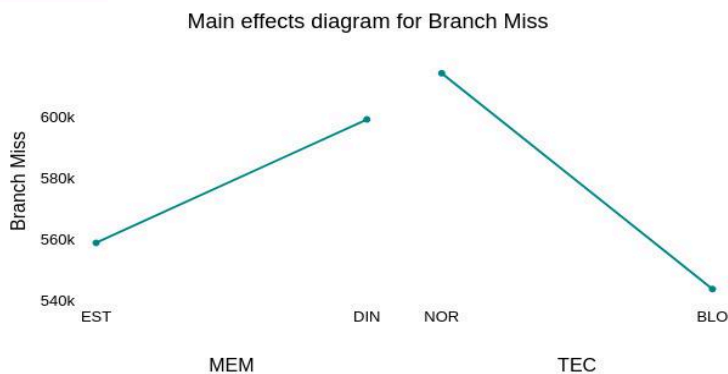
Conclusões

A análise de Branch Instructions, tanto pelo gráfico como pela tabela, mostra que a técnica de otimização (TEC) é o único fator influente, com a blocagem (BLO) reduzindo o número de instruções de desvio em mais de 50%. Este resultado demonstra que a principal vantagem da reestruturação do laço é a diminuição do custo de controle do fluxo de execução. O tipo de alocação de memória (MEM) não apresentou qualquer efeito sobre esta métrica.

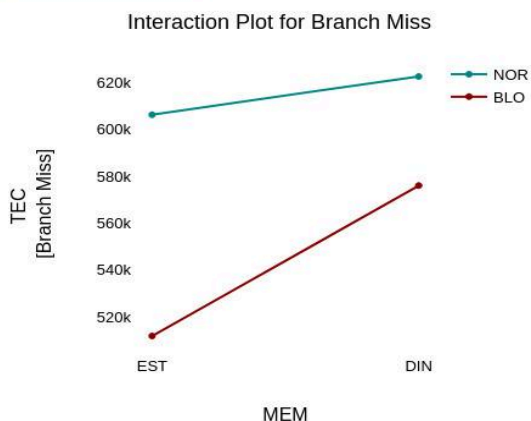
- **Branch-Misses**

Branch Miss		
Parâmetro	Média Estimada	Variação %
Q0	579150,25	
Qa	20108,25	22,63%
Qb	-35215,75	69,40%
Qab	11940,25	7,98%

Main Effects



Interaction Effects



Conclusões

Para a métrica Branch Miss, ambos os fatores mostraram influência, sendo a técnica mais influente do que o tipo de alocação. A técnica de blocagem (BLO) foi eficaz em reduzir os erros de previsão de desvio, tornando o código mais eficiente para o pipeline do processador. Em contrapartida, a alocação dinâmica (DIN) consistentemente piorou essa métrica. O gráfico de interação revela que, embora a blocagem sempre resulte em menos misses que o código normal, a penalidade de usar alocação dinâmica é ligeiramente mais acentuada no código otimizado. Em suma, a otimização de blocagem atua sobre os desvios em duas frentes: primeiro, reduzindo drasticamente a quantidade total de branch instructions a serem executadas e, segundo, tornando o padrão dos desvios restantes mais previsível, o que diminui a taxa de **Branch misses**.